

Servomotor Arduino API Documentation

Version 1.5

Generated: 2026-07-20

Latest Firmware Versions

At the time of generating this API reference, the latest released firmware versions for the servomotors are:

- **Model M17:** servomotor_M17_fw0.15.9.0_scc3_hw1.5.firmware

Table of Contents

1. Hardware Setup
2. Getting Started
3. Data Types
4. Command Reference
5. Basic Control
6. Configuration
7. Device Management
8. Motion Control
9. Other
10. Status & Monitoring
11. Error Handling
12. Error Codes

Hardware Setup

This section covers everything needed to physically connect and power the servomotor before any software is involved.

What you need

- One or more Gearotons servomotors (M17 series: M17-34, M17-40, M17-48, or M17-60).
- A DC power supply providing 12 to 24 V. Budget at least 1.1 A per motor at your chosen voltage (maximum current draw is 1.1 A for the M17-60/48/40 and 1.0 A for the M17-34; rated power 26.4 W for the larger models, 24.0 W for the M17-34).
- A USB-to-RS485 adapter (sold separately; any generic USB-to-RS485 adapter works). The host can be a Mac, PC, Raspberry Pi, Arduino, or ESP32.
- The motor ships with its 6-wire pigtail.

Wiring

Each motor has a 6-pin connector and ships with a matching pigtail wire that plugs into it. The pigtail's six wires, in connector order, are:

- BLACK = GND (power supply negative)
- RED = motor + (power supply positive, +12 to +24 V)
- GREEN = RS485 line A
- BLUE = RS485 line B
- GREEN = RS485 line A (second set)
- BLUE = RS485 line B (second set)

There are two sets of A/B wires so that motors can easily be daisy-chained: one A/B pair comes from the previous device (or the RS485 adapter) and the other A/B pair continues on to the next motor. Electrically the two pairs are the same bus lines.

Connect RED to the supply + terminal and BLACK to the supply - terminal. Connect one GREEN wire to the A terminal and one BLUE wire to the B terminal of the RS485 adapter. Keep each A/B pair twisted if possible.

Grounding: run a common ground between the RS485 adapter's GND terminal and the motors. Because the motors' BLACK power leads are the same ground, tying the adapter's GND to the power supply's negative rail achieves this. A shared ground reference is standard RS485 practice; without one, RS485 communication can be unreliable.

Multiple motors: any number of motors share one bus and one adapter. Either daisy-chain them (adapter A/B into the first motor's first A/B pair, its second A/B pair on to the next motor, and so on) or wire them in a star (all A wires to the adapter's A terminal, all B wires to B) — the daisy chain is what the second A/B pair is for. Power is wired in parallel: all RED leads to supply +, all BLACK leads to supply -. No termination resistors are specified for this product; short benchtop buses typically work without them.

Power supply sizing with many motors: motor inrush and acceleration peaks add up. If many motors may accelerate simultaneously, either size the supply for coincident peaks (about 1.1 A each) or stagger the starts in software (even a random 0-1 s offset per motor spreads the peaks effectively).

Serial link parameters

- Baud rate: 230400 (fixed — there is no autobaud), 8 data bits, no parity, 1 stop bit.
- The bus is half-duplex: only one device transmits at a time. Motion commands execute asynchronously on each motor, so the host can command one motor and immediately talk to others while the first moves.
- Each motor has a factory-programmed 64-bit unique ID and can be assigned a one-byte alias (0-251) for short addressing. Alias 255 is broadcast to all devices.

LED indicators

- GREEN flashing slowly (about once per second): heartbeat — the application firmware is running normally.

- GREEN flashing quickly: the bootloader is running instead of the application (the device is not ready for normal commands; send 'System reset' to relaunch the application).
- GREEN also lights briefly while a packet is being received, so you will see it flicker with communication traffic on the bus.
- RED flashing a repeating count of N blinks with a pause: fatal error number N. Count the blinks or read the code with 'Get status'. An error code of 0 (only possible via a deliberately triggered test) shows the red LED continuously on.

Buttons

The motor has two small buttons:

- Reset: resets the microcontroller. All volatile state (queued moves, zeroed position, limits, enabled state) returns to power-on defaults.
- Test: brief press = spin one way; hold more than 0.3 s and release = spin the other way; hold at least 2 s and release = enter closed loop mode; hold more than 15 s and release = run self-calibration. During calibration the shaft spins and MUST be free to rotate — remove any load first.

Mechanical and environmental

- Standard NEMA 17 mounting: 42.2 x 42.2 mm faceplate with no protrusions. Body heights: M17-60 = 59.7 mm, M17-48 = 48.7 mm, M17-40 = 40.1 mm, M17-34 = 33.5 mm; shaft length 20.6 mm on all models. Weights: 470 / 360 / 285 / 210 g respectively.
- Rated torque: M17-60 = 0.65 N.m, M17-48 = 0.55 N.m, M17-40 = 0.42 N.m, M17-34 = 0.28 N.m. Rated maximum speed 560 RPM for all models (datasheet rating; also the firmware's default max-velocity limit, 9.333 rot/s). Measured unloaded top speed is about 516 RPM (~8.6 rot/s), intrinsic to the drive and the same at 20 V and 24 V; no unit reaches 560 RPM even with a free shaft. In closed-loop operation, commanding a speed above the attainable ceiling grows the tracking error without bound and trips the position-deviation fatal error (45) rather than reaching the speed.
- Built-in magnetic encoder; closed-loop PID control runs on-board at 31.25 kHz.
- Operating temperature 0 to +80 C; storage -20 to +60 C; humidity 20-80% RH non-condensing; IP20 (indoor use).
- Integrated over-voltage and over-temperature protection. Protection trips are fatal errors: the motor disables itself and latches the error code until reset (over-temperature trips at roughly 80 C internal; over-voltage at a firmware-set threshold of 32 V on the M17). Motor current is limited continuously by the firmware-set current limit rather than by a fatal-error trip.
- Regeneration warning: a rapidly decelerating or externally driven motor pumps energy back into the supply and raises the bus voltage. If the supply cannot absorb it, the overvoltage protection (fatal error 14) trips. Decelerate large inertias gently, and avoid spinning the shaft forcefully by hand while powered.

Getting Started

This example demonstrates a trapezoid move with the servomotor:

```
// Minimal Arduino example: Trapezoid move using built-in unit conversions
// Goal: spin the motor exactly 1 rotation in 1 second, then stop.
// Sequence:
//   enable MOSFETs -> trapezoidMove(1.0 rotations, 1.0 seconds) -> wait 1.1s -> disable MOSFETs.
//
// Notes:
// - This uses the library's unit conversion (no raw counts/timesteps).
// - Configure Serial1 pins for your board (ESP32 example pins below).
// - Motor is created AFTER Serial1.begin(...) so hardware UART pins are set first.

#include <Servomotor.h>

#define ALIAS 'X' // Device alias
#define BAUD 230400 // RS485 UART baud rate
#define DISPLACEMENT_ROTATIONS 1.0f // 1 rotation
#define DURATION_SECONDS 1.0f // 1 second
#define TOLERANCE_PERCENT 10 // +10% wait margin because the motor's clock is not
// perfectly accurate
#define WAIT_MS ((unsigned long)(DURATION_SECONDS * 1000.0f * (100 + TOLERANCE_PERCENT) / 100))

// Example RS485 pins for ESP32 DevKit (change as needed for your board)
#if defined(ESP32)
#define RS485_TXD 4 // TX pin to RS485 transceiver
#define RS485_RXD 5 // RX pin from RS485 transceiver
#endif

void setup() {
  Serial.begin(115200); // Console serial for debugging

  // Create the motor; serial port opens on first instantiation.
#if defined(ESP32)
  Servomotor motor(ALIAS, Serial1, RS485_RXD, RS485_TXD);
#else
  Servomotor motor(ALIAS, Serial1);
#endif

  // Use units: rotations for position, seconds for time
  motor.setPositionUnit(PositionUnit::SHAFT_ROTATIONS);
  motor.setTimeUnit(TimeUnit::SECONDS);

  motor.enableMosfets();
  motor.trapezoidMove(DISPLACEMENT_ROTATIONS, DURATION_SECONDS);
  delay(WAIT_MS);
  motor.disableMosfets();
}

void loop() {
}
```

Data Types

This section describes the various data types used in the Servomotor API commands.

Integer Data Types

Type	Size (bytes)	Range	Description
i16	2	-32,768 to 32,767	16-bit signed integer
i32	4	-2,147,483,648 to 2,147,483,647	32-bit signed integer
i48	6	-549,755,813,888 to 549,755,813,887	48-bit signed integer
i64	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	64-bit signed integer
i8	1	-128 to 127	8-bit signed integer
u16	2	0 to 65,535	16-bit unsigned integer
u32	4	0 to 4,294,967,295	32-bit unsigned integer
u48	6	0 to 1,099,511,627,775	48-bit unsigned integer
u64	8	0 to 18,446,744,073,709,551,615	64-bit unsigned integer
u8	1	0 to 255	8-bit unsigned integer

Special Data Types

Type	Size (bytes)	Description
buf10	10	10 byte long buffer containing any binary data
crc32	4	32-bit CRC
firmware_page	2058	This is the data to upgrade one page of flash memory. Contents includes the product model code (8 bytes), firmware compatibility code (1 byte), page number (1 byte), and the page data itself (2048 bytes).
general_data	Variable	This is some data. You will need to look elsewhere at some documentation or into the source code to find out what this data is.
list_2d	Variable	A two dimensional list in a Python style format, for example: [[1, 2], [3, 4]]
string8	8	8 byte long string with null termination if it is shorter than 8 bytes

string_null_term	Variable	This is a string with a variable length and must be null terminated
success_response	Variable	Indicates that the command was received successfully and is being executed. The next command can be immediately transmitted without causing a command overflow situation.
u24_version_number	3	3 byte version number. the order is patch, minor, major
u32_version_number	4	4 byte version number. the order is development number, patch, minor, major
u64_unique_id	8	The unique ID of the device (8-bytes long)
u8_alias	1	This can hold an ASCII character where the value is represented as an ASCII character if it is in the range 33 to 126, otherwise it is represented as a number from 0 to 255
unknown_data	Variable	This is an unknown data type (work in progress; will be corrected and documented later)

Command Reference

Basic Control

Disable MOSFETs

Immediately disable the motor driver outputs (MOSFETs); the command executes at once (it is not queued) and is idempotent. Note that the MOSFETs are disabled by default after power-on, and any fatal error also force-disables them. Disabling does NOT stop or clear the movement queue: queued moves keep executing virtually with the outputs off, so the commanded position keeps advancing while the rotor stands still, and because the position-deviation check stays armed, disabling mid-move typically ends in fatal error `ERROR_POSITION_DEVIATION_TOO_LARGE` once the deviation exceeds the limit (default 2 shaft rotations), requiring a 'System reset' to recover. Movement commands continue to be accepted (with success responses) while the MOSFETs are disabled. Takes no parameters; any payload bytes raise fatal error `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Disable MOSFETs
motor.disableMosfets();
```

Enable MOSFETs

Enable the motor driver outputs (MOSFETs). Executes immediately (not queued). On M1, M2, and M23, enabling first drives the phases to a zero-volt state, samples the motor current-sensor baseline, raises fatal error `ERROR_CURRENT_SENSOR_FAILED` if the baseline is outside the expected window, and calibrates the overcurrent watchdog thresholds from that baseline; M17 performs no current-sensor check. Calling while already enabled is a no-op (the baseline check is skipped) and still returns success. Enabling does not re-align the commanded position to the measured hall position and clears no queue or PID state; instead the rotor mechanically snaps to the angle derived from the commanded position, a settling transient lasting roughly 0.3 seconds (on M17 the external stepper driver additionally slews at most 1 microstep per 32 microsecond control tick). The position-deviation check is armed throughout, so applying tight deviation limits before this transient settles can trip fatal error `ERROR_POSITION_DEVIATION_TOO_LARGE`; the safe sequence is enable, wait to settle, 'Zero position', tighten limits, then move. Takes no parameters; any payload bytes raise fatal error `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Enable MOSFETs
motor.enableMosfets();
```


Reset time

Reset the device's absolute microsecond clock to zero and, as a major side effect, clears the entire movement queue and stops the motor instantly: all queued moves are silently discarded, the current velocity is forced to zero immediately (no controlled deceleration), the queue's expected end position is snapped to the current position, and the debug profiler timestamps are reset. Unlike 'Emergency stop', the MOSFETs stay enabled and the motor holds position. Do not send this mid-motion unless an abrupt halt is intended. It executes immediately (not queued) and the success response only confirms the reset occurred. On broadcast the reset is still executed by every device and only the response is suppressed; broadcasting is the intended way to zero all motors' clocks simultaneously before using 'Time sync' or 'Multimove'. Call it before issuing timed movement commands so device time starts from a known epoch. The command takes no payload; any nonzero payload raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, which disables the MOSFETs and halts the device until 'System reset'.

Example:

```
// Reset time
motor.resetTime();
```

Emergency stop

Stop the motor immediately: disables the MOSFETs, clears the movement queue, zeroes the current velocity, and snaps the internal position target to the current position. This command executes at once (it is not queued). The success response confirms the stop has already been performed. Afterward the motor has no holding torque and a load can backdrive it; send 'Enable MOSFETs' before commanding motion again. The position-deviation watchdog stays armed, so backdriving the shaft more than the max allowable position deviation (default 2 shaft rotations) raises fatal error 45, `ERROR_POSITION_DEVIATION_TOO_LARGE`, even though the motor is unpowered. In firmware before 0.15.4.0 the motion planner's predicted end-of-queue velocity was not reset by the queue clear, so if the stop interrupted motion, later queued moves could raise spurious fatal errors (28, `ERROR_PREDICTED_VELOCITY_TOO_HIGH`; 27, `ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE`; 26, `ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE`) or land at the wrong physical position unless 'System reset' was sent first; firmware 0.15.4.0 and later resets it correctly (verified on hardware) and needs no reset before queueing further moves. Broadcasting this command stops all motors at once (each executes silently). Any payload bytes raise fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Emergency stop
motor.emergencyStop();
```

Zero position

Make the current position the zero position (the origin). Executes immediately and atomically zeroes the commanded position, the end-of-queue target position, the hall-sensor measured position, the current velocity, and the full PID state (integral term and error history), so 'Get position' and 'Get hall sensor position' both read approximately zero afterward and the motor does not move or jerk. Works with MOSFETs enabled or disabled, in open or closed loop, but the movement queue must be empty, otherwise the command raises fatal error 8, `ERROR_QUEUE_NOT_EMPTY`, and no success response is sent. Warning: safety limits are not rebased; limits set earlier with 'Set safety limits' refer to different physical locations after zeroing, and if the old limit window does not contain zero the next control cycle raises fatal error 25, `ERROR_SAFETY_LIMIT_EXCEEDED`, so set safety limits only after zeroing (or set them again). Zeroing also discards any accumulated PID wind-up. (In firmware before 0.15.4.0 it did not repair the stale planner velocity left by a mid-motion 'Emergency stop'; 0.15.4.0 fixed that staleness at the source.) Any payload bytes raise fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Zero position
motor.zeroPosition();
```

System reset

Reset the device. A success response is sent and fully transmitted before the reset occurs, so the host will see the reply; when broadcast, all addressed devices reset and no reply is sent. After the reset the device runs its bootloader for a 250 ms window before automatically launching the application firmware. Any valid-CRC packet addressed to the device (by alias, unique ID, or broadcast) received during that window permanently cancels the launch and keeps the device in the bootloader until another 'System reset' or a power cycle; packets not addressed to the device, or with a bad CRC, do not cancel it. So after resetting, either send nothing for well over 250 ms or deliberately send a command to stay in the bootloader (for example for a firmware upgrade). This is the only software way to clear a latched fatal error; together with 'Get status' it is one of only two commands accepted while the device is in a fatal-error state. The bootloader implements this command with identical semantics, so it can also bounce a device pinned in the bootloader back into the application. If no valid application firmware is present (its CRC check fails), the device stays in the bootloader indefinitely. Sending any payload bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, instead of resetting.

Example:

```
// System reset
motor.systemReset();
```

Configuration

Set maximum velocity

Set the maximum allowed velocity. This value IS used (the old note claiming otherwise is wrong), in three ways: it sizes the acceleration ramp of every 'Trapezoid move' and 'Go to position' (ramp time equals max velocity divided by max acceleration); it validates every queued move, raising fatal error `ERROR_PREDICTED_VELOCITY_TOO_HIGH` if the move would exceed it; and it is enforced against the live commanded velocity on every 32 microsecond control tick, so it takes effect immediately, even on moves already queued or in flight. WARNING: lowering the limit below the velocity of an in-flight move raises fatal error `ERROR_VEL_TOO_HIGH` on the next tick, halting the device until 'System reset'. As of firmware 0.15.6.0, a value of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (firmware through 0.15.5.0 accepted it without complaint, after which every 'Trapezoid move'/'Go to position' was silently planned as a zero-motion dwell -- success response, queue occupied for the commanded duration, but the shaft never moved and no error was raised); note that any requested limit below half an internal unit rounds down to 0 during unit conversion, so tiny limits can hit this rejection unexpectedly. The limit comparison is inclusive: a move at exactly the limit is accepted. As of firmware 0.15.8.0 the boot default is the datasheet maximum speed: 560 RPM (9.333 rotations/second), slightly above the ~8.6 rot/s an unloaded motor actually reaches, so out of the box the motor itself -- not this limit -- is what tops out. The limit may be RAISED for experimentation up to a clamp of twice the default (18.67 rot/s; higher requested values are silently clamped with no error or feedback) or LOWERED freely, and enforcement follows immediately (a move commanding more than the limit is rejected with fatal error 16, or 28 for predicted velocities). History: 0.15.7.0 briefly defaulted to the 34 rot/s electrical rating; firmware before that booted at 68 rot/s due to a constant derivation bug. Executes synchronously, not queued. The payload is a single u32 (4 bytes); any other size raises fatal error `ERROR_COMMAND_SIZE_WRONG`.

Parameters:

- *maximumVelocity*: u32: Maximum velocity limit. Raw-packet users: the wire value is in internal velocity units, i.e. encoder counts per 32 microsecond timestep multiplied by 2^{20} ; sending literal counts per timestep sets a limit 2^{20} times too small. Values above the firmware ceiling `MAX_VELOCITY` are silently clamped. (Library versions before the 2026-07 fix carried a wrong conversion factor, about 104.86 times too large, for the selectable unit named 'counts_per_timestep'; the current library is correct.)

Example:

```
// Set maximum velocity
uint32_t maximumVelocity = 1000;

motor.setMaximumVelocity(maximumVelocity);
```

Set maximum acceleration

Set the maximum acceleration used to plan all trapezoid moves queued afterwards ('Go to position', 'Trapezoid move', calibration moves); items already in the queue are unaffected. The same value is the fatal-error threshold for 'Move with acceleration': a queued item exceeding it raises fatal error 15, `ERROR_ACCEL_TOO_HIGH`. Values above the firmware clamp are silently clamped to the clamp and success is still returned. The setting is RAM-only and reverts to the default on any reset or power cycle. A value of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE`, because trapezoid planning divides by this setting (firmware before 0.15.4.0 accepted 0 and later divided by zero). Raw-packet note: the wire value is encoder counts per timestep squared in Q24 fixed point, that is the counts/timestep² value multiplied by 2²⁴; the library's unit conversion applies this factor automatically. As of firmware 0.15.8.0 the boot default is 12000 rotations/second², matching the product spec and slightly above the ~11000 rot/s² measured unloaded spin-up, so out of the box the motor itself is the practical limit. The limit may be RAISED up to a clamp of 48000 (higher requested values are silently clamped with no error) or LOWERED freely, and enforcement follows immediately (a move demanding more is rejected with fatal error 15). History: 0.15.7.0 briefly defaulted to 500 with a clamp of 2000; earlier firmware booted at 2000 due to a constant derivation bug.

Parameters:

- *maximumAcceleration*: u32: The maximum acceleration. On the wire this is encoder counts per timestep squared in Q24 fixed point (the counts/timestep² value multiplied by 2²⁴); the library's unit conversion applies this factor. Values above the firmware cap are silently clamped to the cap. A value of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (firmware before 0.15.4.0 accepted it and later divided by zero during trapezoid planning).

Example:

```
// Set maximum acceleration
uint32_t maximumAcceleration = 1000;

motor.setMaximumAcceleration(maximumAcceleration);
```

Start calibration

Run a full motor calibration. The device spins the shaft about 1.5 rotations back and forth for 6 cycles (several seconds per cycle, product dependent: about 3.5 s on M17 and 4 s on M23) while measuring the hall-sensor midlines and determining the motor phase direction and commutation position offset, then writes these settings to flash and automatically reboots the MCU. The shaft must be free to rotate. The command resets the driver IC, enables the MOSFETs itself, and zeroes the current position and commutation offset, discarding any prior 'Zero position'. The success response is sent before calibration runs and acknowledges only that it is starting. Completion is signaled solely by the reboot, which erases all volatile state (zeroed position, maximum velocity and acceleration, enabled MOSFETs); wait about 2 seconds after the reboot before sending any command or the device may be pinned in its bootloader. Preconditions, each raising a fatal error if violated: 7 `ERROR_NOT_IN_OPEN_LOOP` if not in open-loop position control, 8 `ERROR_QUEUE_NOT_EMPTY` if the movement queue is not empty, 19 `ERROR_MOTOR_BUSY` if already calibrating or homing, 41 `ERROR_TEST_MODE_ACTIVE` if a test mode is active. Any movement command sent during calibration raises fatal error 19, `ERROR_MOTOR_BUSY`. If a hall channel produces too few minima or maxima the device halts with fatal error 11, `ERROR_NOT_ENOUGH_MINIMA_OR_MAXIMA`, and never reboots, so treat the absence of a reboot within the expected time as failure. The payload must be empty. Broadcasting starts calibration on every addressed device silently and leaves the whole bus unresponsive until the devices reboot.

Example:

```
// Start calibration
motor.startCalibration();
```

Set maximum motor current

Set the maximum motor current and maximum regeneration current. Executes immediately (not queued); the success response acknowledges the values are already applied. The values are not saved to non-volatile memory and revert to the product-specific firmware default after a reset (M1: 100, M2/M17/M23: 200). Units are arbitrary: internally the motor current value caps the applied PWM duty ('PWM voltage'), limiting current only indirectly, and it also rescales the PID controller's authority (maximum error, integral limit, and output limit are derived from it); setting it while the motor is moving clamps the integral term and zeroes the derivative-filter state, so expect a brief control transient. Both parameters are validated against a product-specific ceiling (M1/M2: 300, M17: 390, M23: 1024); exceeding it with EITHER parameter raises fatal error 23, `ERROR_MAX_PWM_VOLTAGE_TOO_HIGH`, which disables the MOSFETs until 'System reset'. Do not choose values above the product ceiling.

Parameters:

- *motorCurrent*: u16: The maximum motor current in arbitrary units (not amps). A value of 150 or 200 is suitable. Internally this caps the applied PWM duty and rescales the PID controller's authority; at speed the firmware additionally allows back-EMF compensation on top of this cap, so it acts as a torque/current cap rather than an absolute duty limit. Must not exceed the product-specific maximum (M1/M2: 300, M17: 390, M23: 1024) or fatal error 23, `ERROR_MAX_PWM_VOLTAGE_TOO_HIGH`, is raised.
- *regenerationCurrent*: u16: The motor regeneration current (while braking) in the same arbitrary units. It sets the regen-side analog-watchdog threshold, which currently only drives the red LED on M1/M2 and has no functional effect on M17/M23. The value is still range-checked: exceeding the product-specific maximum (M1/M2: 300, M17: 390, M23: 1024) raises fatal error 23, `ERROR_MAX_PWM_VOLTAGE_TOO_HIGH`, so do not treat this parameter as ignorable.

Example:

```
// Set maximum motor current
uint16_t motorCurrent = 100;
uint16_t regenerationCurrent = 100;

motor.setMaximumMotorCurrent(motorCurrent, regenerationCurrent);
```

Set safety limits

Set the position safety limits. Takes effect immediately (not queued); the success response acknowledges the limits are already applied. Enforcement is fatal, not clamping: every motor-control tick the actual position is compared against the limits, and a position outside them raises fatal error 25, `ERROR_SAFETY_LIMIT_EXCEEDED`, disabling the MOSFETs until 'System reset'. Consequently, sending limits that exclude the `CURRENT` position faults the device essentially instantly. Queued moves are also pre-checked at enqueue time: a predicted final position or velocity-reversal turn point outside the limits raises fatal error 27, `ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE`, or fatal error 26, `ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE`, rather than the move being trimmed. The limits are compared in the firmware's internal position units (encoder counts, the same domain used by 'Get position' and 'Go to position'), not microsteps. As of firmware 0.15.5.0, a lower limit greater than the upper limit is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE`. WARNING for firmware up to and including 0.15.4.0: inverted limits were stored unvalidated, making every position out of bounds and faulting the device (error 25) on the next control tick -- and because that fatal error fires while this command's own success reply is still transmitting, it can truncate the reply and, depending on the exact timing, deadlock the device completely (no response to any command, not even 'Get status' or 'System reset') until power cycle; on such firmware never send lower > upper. (0.15.5.0 also fixed the underlying race for all fatal errors: a fatal that fires while any reply is transmitting no longer truncates it or hangs the device.) Also note the limits are interpreted in the current position frame: 'Zero position' shifts which physical locations they refer to, so re-send the limits after zeroing whenever the fence protects real hardware. The limits are not saved to non-volatile memory; a reset restores `INT64_MIN/INT64_MAX`, effectively disabling them.

Parameters:

- *lowerLimit*: i64: The lower position limit in the firmware's internal position units (encoder counts, the same domain returned by 'Get position'), not microsteps.
- *upperLimit*: i64: The upper position limit in the firmware's internal position units (encoder counts), not microsteps. Must be greater than or equal to the lower limit; as of firmware 0.15.5.0 swapped limits are rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (firmware through 0.15.4.0 stored them unvalidated and faulted the device immediately with error 25).

Example:

```
// Set safety limits
float lowerLimit = 0;
float upperLimit = 0;

motor.setSafetyLimits(lowerLimit, upperLimit);
```

Test mode

Set or trigger a test mode. Executes immediately (not queued); intended only for developers and production testing. As of firmware 0.15.4.0, value 0 safely clears all test modes (motor test mode, overtemperature test mode, and the overvoltage-threshold override). WARNING for firmware before 0.15.4.0: value 0 hangs the device forever before any reply and only a power cycle recovers it -- on older firmware clear test modes only with 'System reset'. Of motor test modes 1-9, only 2, 3 and 4 do anything: 2 suppresses the fatal error on a failed 'Go to closed loop' (the device drops back to open loop with MOSFETs disabled instead), 3 captures one PID debug snapshot (error, P/I/D terms, output) into the multipurpose buffer (read it with 'Read multipurpose buffer'), and 4 formerly read the GC6609 driver registers into that buffer, but is DEFUNCT on current units (the GC6609 is used only on legacy M17 units with software compatibility code 1; on current hardware the buffer stays empty and the mode stays latched until reset). Any nonzero motor test mode makes 'Start calibration' raise fatal error `ERROR_TEST_MODE_ACTIVE`. LED test modes 10-13 (10=both LEDs off, 11=green on, 12=red on, 13=both on) send the success response and then hang forever; power cycle required. Values 14-73 deliberately trigger fatal errors 0-59 (value minus 14): no success response is sent -- the device replies with an error packet and stays in the fatal-error state until 'System reset'. Value 14 raises fatal error 0, which shows a solid red LED while the status error code reads 0 and is easily mistaken for no error. Value 74 sets the overvoltage-protection threshold to 22 V (production test; should trip on a 24 V supply), 75 sets it to 26 V (should not trip on 24 V), and 76 raises the overtemperature cutoff to about 90 degrees C (default about 80); these take effect immediately and only a reset restores the defaults. Values above 76 raise fatal error 53, `ERROR_INVALID_TEST_MODE`. Values 0-9 and 74-76 return a success response, as do LED modes 10-13 (which hang immediately after sending it); values 14-73 and values above 76 reply with an error packet instead. Broadcast executes with no reply; broadcasting 10-13 bricks every device on the bus until power cycled (broadcasting 0 does too, but only on firmware before 0.15.4.0).

Parameters:

- *testMode*: u8: The test mode to use or trigger. See the command description for what each value range does. Never send 10-13: they hang the device until it is power cycled. Value 0 safely clears all test modes on firmware 0.15.4.0 and later (on older firmware value 0 also hangs the device). Values 14-73 trigger fatal errors and values above 76 raise fatal error 53, `ERROR_INVALID_TEST_MODE`.

Example:

```
// Test mode
uint8_t testMode = 1;

motor.testMode(testMode);
```

Set PID constants

Set the P, I, and D constants of the closed-loop controller that tries to maintain the motion trajectory. Executes immediately, not queued: the new constants are applied atomically (interrupts briefly disabled) before the success response is sent, even mid-move, which can cause a brief control transient. The values are raw fixed-point gains: the controller output is ($P \cdot \text{error} + \text{integral term} + \text{derivative term}$) right-shifted by a product-specific amount (18 on M1/M2, 11 on M17, 14 on M23), so identical numbers behave very differently across products. The usable range of each constant is 0 to 2147483647. Setting the constants also rescales internal safety limits: the maximum usable error window is derived as output authority divided by P (a very large P shrinks it), and the integral accumulator is clamped to 25 percent of output authority for anti-windup. The constants are not persisted: a reset or power cycle restores compiled-in per-product defaults (P/I/D: M1 5000/1/5000, M2 20000/1/100000, M17 2000/5/175000, M23 500/1/1000). The command has no preconditions and is accepted in any state; a payload other than exactly 12 bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Parameters:

- *kP*: u32: The proportional term constant (P), a raw fixed-point gain. Usable range is 0 to 2147483647; do not exceed this.
- *kI*: u32: The integral term constant (I), a raw fixed-point gain. Usable range is 0 to 2147483647; the firmware may silently cap large values further for overflow safety.
- *kD*: u32: The differential term constant (D), a raw fixed-point gain. Usable range is 0 to 2147483647. The firmware right-shifts this value by 5 bits internally, so values 0 to 31 produce no derivative action and D is effectively quantized in steps of 32.

Example:

```
// Set PID constants
uint32_t kP = 1000;
uint32_t kI = 1000;
uint32_t kD = 1000;

motor.setPidConstants(kP, kI, kD);
```


Set max allowable position deviation

Set the maximum distance that the actual motor position (as measured by the hall sensors) is allowed to deviate from the commanded position; exceeding it raises fatal error 45 (ERROR_POSITION_DEVIATION_TOO_LARGE). Takes effect immediately (not queued). The firmware stores the absolute value of the input, so negative values behave as positive; as of firmware 0.15.9.0 the value INT64_MIN (which has no positive counterpart) saturates to the maximum limit. (Firmware 0.15.4.0 through 0.15.8.0 had a landmine here: INT64_MIN made the stored limit go negative and fatal error 45 latched on the very next control tick.) The setting is RAM-only and reverts to the default of 2 shaft rotations (6553600 microsteps on M3, M17, and M23; one encoder count equals one internal microstep) at power-up or after 'System reset'. The deviation check runs continuously in the background except during calibration, and it is not gated on the MOSFETs being enabled or a move being in progress: externally turning the shaft past the limit while the motor is idle or disabled also trips fatal error 45. When the error trips, the signed deviation, absolute deviation, commanded position, and hall sensor position are latched into debug values 1 to 4, readable via 'Get debug values'. A payload that is not exactly 8 bytes raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Parameters:

- *maxAllowablePositionDeviation*: i64: The new maximum allowable position deviation; the firmware stores the absolute value, so negative values behave as positive

Example:

```
// Set max allowable position deviation
float maxAllowablePositionDeviation = 0;

motor.setMaxAllowablePositionDeviation(maxAllowablePositionDeviation);
```

CRC32 control

Enable or disable the CRC32 layer of the protocol for this device, in both directions. When enabled (the power-up default), every received packet must carry a trailing 4-byte CRC32 and is validated against it, and every response is sent with response-indicator byte 253 followed by a CRC32; when disabled, received packets must not carry a CRC32 and responses use indicator byte 252 with no CRC32. The change takes effect immediately, before the acknowledgment is transmitted, so the success response to this very command already arrives in the new format. The setting is RAM-only and reverts to enabled at every power-up or 'System reset'; a host that disables CRC32 must re-send this command after any reset. While CRC32 is enabled, a received packet that fails validation is dropped silently with no error response: the failure appears only as a timeout and an incremented counter readable via 'Get communication statistics'. Broadcasting applies the change to every device on the bus with no responses, which is the practical way to switch a whole bus at once. A payload that is not exactly 1 byte raises fatal error 51, ERROR_COMMAND_SIZE_WRONG.

Parameters:

- *enableCrc32*: u8: Control value: 0 disables the CRC32 protocol, any nonzero value enables it

Example:

```
// CRC32 control
uint8_t enableCrc32 = 1;

motor.crc32Control(enableCrc32);
```

Device Management

Time sync

Send the master's clock reading to the motor so the motor can discipline its clock rate. The motor never sets or steps its clock to the sent value: it computes the signed error (master minus local, clamped to plus or minus 32768 microseconds per call) and feeds it to a PI controller that only re-trims the internal HSI16 oscillator via the HSITRIM field of the RCC-ICSCR register, clamped to the narrow range 62 to 66 (center 64). Drift therefore converges gradually; a large initial offset is never corrected by this command, so establish a shared epoch first, for example by broadcasting 'Reset time'. Send it regularly, roughly 10 times per second, to each motor. Broadcast is a complete no-op: on a broadcast packet the oscillator is not trimmed and no reply is sent, so each motor on a shared bus must be addressed individually. Executes immediately (not queued). The payload must be exactly 4 bytes or the device raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`; a malfunctioning clock can also raise fatal error 1, `ERROR_TIME_WENT_BACKWARDS`.

Parameters:

- *masterTime*: u32: The low 32 bits of the master's absolute clock, in microseconds (this 32-bit window wraps every ~71.6 minutes). The device truncates its own 64-bit clock to 32 bits and subtracts modularly, so the computed error is only correct while the true offset between master and device is under about 35.8 minutes -- hence the need to sync regularly.

Example:

```
// Time sync
uint32_t masterTime = 1000;

timeSyncResponse response = motor.timeSync(masterTime);
```

Get product specs

Get two product constants needed for motion math: the update frequency in Hz (the rate of the control loop that runs the hall sensor position, movement, PID, and safety calculations) and the number of position counts per one shaft rotation. One time step, as used by 'Move with acceleration', 'Move with velocity', and the other movement commands, is exactly $1/\text{updateFrequency}$ seconds (31250 Hz, i.e. 32 microseconds, on current firmware). `countsPerRotation` is product-specific (3276800 for M3/M17/M23, 639744 for M1, 4569600 for M2), so always query it rather than hardcoding a value. Sending any payload bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Get product specs
getProductSpecsResponse response = motor.getProductSpecs();
```

Detect devices

Detect all devices connected on the RS485 interface. Normally sent to the broadcast address (255); every device replies exactly once with its unique ID and current alias, each after its own pseudo-random delay of 0 to 950 ms, so listen for the full one-second window. A collision between replies is possible but unlikely; repeat the command if devices you expect were not discovered. Even when addressed to one specific device by alias or unique ID, the reply still arrives only after the random delay. Warning: for about 1 second after receiving this command, a device silently discards every incoming packet (no error, no response, nothing queued), so wait at least about 1.1 seconds after sending it before transmitting anything else, including a repeated 'Detect devices'. Devices currently sitting in the bootloader also answer this command, so it discovers devices in either mode.

Example:

```
// Detect devices
detectDevicesResponse response = motor.detectDevices();
```

Set device alias

Set the device's one-byte alias, save it to nonvolatile flash, and then automatically reset the device. Valid aliases are 0 to 251 and 255; setting 255 removes the alias (255 is the broadcast address, so afterwards unique-ID extended addressing is the only way to address the device individually). Values 252 to 254 are reserved and raise fatal error 50, `ERROR_BAD_ALIAS`: an error packet is sent instead of the success response and the device is left with MOSFETs disabled, answering only 'Get status' and 'System reset' until reset. On success the response is transmitted first, in reply to the old address; then the settings are written to flash and the device reboots itself. It drops off the bus briefly while it saves to flash and reboots (bench-measured: with a SILENT bus it answered in application mode 0.3 s after the command; wait at least 0.5 s of bus silence before addressing it at the new alias -- continuous polling during the reboot delays recovery to over 1.2 s and risks pinning the device in its bootloader window) and comes back under the new alias with all volatile state (motion queue, position zeroing, enabled MOSFETs) lost, so pause before sending the next command. Warning: broadcasting this command assigns the same alias to every device on the bus with no response sent, and a broadcast with a reserved alias silently locks every device in the fatal-error state. The command also works while the device is in the bootloader, with the same save-and-reset behavior.

Parameters:

- *alias*: `u8_alias`: The alias (which is a one byte ID) ranging from 0 to 251. Values 252 to 254 are reserved and raise fatal error 50, `ERROR_BAD_ALIAS`. You can set it to 255, which will remove the alias; the device is then addressable individually only by its unique ID.

Example:

```
// Set device alias
uint8_t alias = 1;

motor.setDeviceAlias(alias);
```

Get product info

Get the product information: product code (model number), firmware compatibility code, hardware version, serial number, and unique ID, returned as a 28-byte response payload. This is an immediate query with no side effects. Before a firmware upgrade, read the product code and firmware compatibility code from the target device: they are exactly the values the bootloader checks against the header of every page sent with 'Firmware upgrade', and they must match the firmware file. The command is also serviced while the device is in the bootloader; the data is stored in bootloader flash, independent of the application image. No response is sent when the command is broadcast. The command takes no payload; sending any payload bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, which disables the MOSFETs and locks the device until 'System reset'.

Example:

```
// Get product info
getProductInfoResponse response = motor.getProductInfo();
```

Firmware upgrade

Write one 2048-byte page of application firmware to flash. This command only works while the device is running the bootloader: the main firmware silently discards it with no response, so the host just times out. To upgrade, reset the device ('System reset' or power cycle), then send any valid packet within the bootloader's 250 ms launch window; this cancels the pending application launch and holds the device in the bootloader indefinitely (confirm via 'Get status' bit 0 or 'Get firmware version'), and send 'System reset' when finished to boot the new firmware. Each page's model code and firmware compatibility code must match the device's 'Get product info' values; a mismatch, or a payload shorter than 9 bytes, is dropped with no response at all, indistinguishable from a dead device. A total payload other than 2058 bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`; a page number outside 5 to 30 raises fatal error 52, `ERROR_INVALID_FLASH_PAGE`. The success response is sent only after the page is erased and burned, so pages can be streamed back-to-back, one per acknowledgment. The application image must begin with a u32 count of its 32-bit words, followed by the code, with the CRC32 of those words stored immediately after; the bootloader validates this on every boot and will not launch an image that fails, leaving the device stuck in the bootloader. Broadcasting suppresses the per-page responses but still writes flash, which allows flashing multiple identical devices at once (devices with a non-matching model code silently ignore the pages).

Parameters:

- *firmwarePage*: `firmware_page`: The data to upgrade one page of flash memory (2058 bytes total). Contents: the product model code (8 bytes), firmware compatibility code (1 byte), absolute flash page number (1 byte), and the page data itself (2048 bytes). Valid page numbers are 5 through 30; page 5 is the start of the application area at 0x8002800 (pages 0 to 4 hold the bootloader and page 31 holds settings). Any other page number raises fatal error 52, `ERROR_INVALID_FLASH_PAGE`.

Example:

```
// Firmware upgrade
float firmwarePage = 0;

motor.firmwareUpgrade(firmwarePage);
```

Get product description

Get a short product description string. The device replies immediately with a fixed, compile-time null-terminated string (in the current main firmware this is 'Servomotor'); the null terminator is included in the response. This is an immediate query with no effect on motion or device state, and it works both in the main firmware and in the bootloader. The command takes no payload; sending any payload bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, which disables the MOSFETs and latches until 'System reset'. When broadcast (address 255) the command produces no response at all, so it is only useful when addressed to a single device.

Example:

```
// Get product description
getProductDescriptionResponse response = motor.getProductDescription();
```

Get firmware version

Get the firmware version, or the bootloader version if the device is currently running the bootloader. The reply payload is exactly five bytes: four version bytes in the order development, patch (bugfix), minor, major, followed by a dedicated one-byte flag that is 1 when the reply comes from the bootloader and 0 when it comes from the main firmware. This flag byte is not the device status bitfield returned by 'Get status'; it is a separate byte in which no other bits are ever set. When the device is in the bootloader, the four version bytes are the bootloader's own version, not the application firmware's. The command takes no payload; sending any payload bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`. When broadcast, the command sends no response, in both the firmware and the bootloader.

Example:

```
// Get firmware version
getFirmwareVersionResponse response = motor.getFirmwareVersion();
```

Ping

Send a 10-byte payload and the device immediately responds with the same 10 bytes (executes at once, not queued). The payload must be EXACTLY 10 bytes: any other size raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, which disables the MOSFETs until 'System reset', so a malformed ping halts the device. A broadcast ping is processed but produces no response at all, so it cannot be used for bus discovery; use 'Detect devices' for that.

Parameters:

- *pingData*: buf10: The binary data payload to send to the device. Must be exactly 10 bytes; any other length raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Ping
float pingData = 0;

pingResponse response = motor.ping(pingData);
```

Vibrate

Cause the motor to vibrate (implemented only on the M1 product; a silent no-op on M2, M17, and M23) by rapidly alternating the open-loop PWM voltage between +50 and -50 (a fixed, hard-coded amplitude), or stop vibrating. This is implemented only on the M1 product: on M2, M17, and M23 the command does nothing but still returns a normal success response, so the caller cannot tell that it had no effect. The command executes immediately, not through the motion queue. On M1, sending it while the motion queue is not empty raises fatal error 8, `ERROR_QUEUE_NOT_EMPTY`; this check runs before the turn-off logic, so even sending value 0 faults if moves are queued. Turning vibration on while the motor is busy raises fatal error 19, `ERROR_MOTOR_BUSY`, and while vibrating the motor is marked busy, so other motion commands will fault until vibration is turned off with value 0. Turning vibration on implicitly enables the MOSFETs (which can raise fatal error 22, `ERROR_CURRENT_SENSOR_FAILED`), switches the control mode to open-loop position control, and overwrites the shared buffer read by 'Read multipurpose buffer'. Turning it off leaves the MOSFETs enabled and the control mode unchanged. On all products the payload must be exactly 1 byte or fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, is raised.

Parameters:

- *vibrationLevel*: u8: Vibration control: 0 turns vibration off; any nonzero value turns it on. The value is only tested for zero versus nonzero, and the vibration amplitude is fixed in firmware, so the number does not scale intensity. Effective only on the M1 product; other products accept the command but do nothing.

Example:

```
// Vibrate
uint8_t vibrationLevel = 1;

motor.vibrate(vibrationLevel);
```

Identify

Identify a motor by flashing its green LED rapidly: 30 flashes at roughly 90 ms each, about 2.7 seconds total, after which the normal 1-second heartbeat blink resumes. The command executes immediately and is purely cosmetic: it does not touch the motion queue, motion state, or MOSFETs, so it is safe to send mid-move. Sending it again while flashing restarts the 30-flash sequence from the beginning. When unicast, the success response is sent after the flashing has started. Broadcasting to address 255 makes every motor on the bus flash with no replies, which is the intended way to visually identify all connected motors. The command takes no payload and there is no unique-ID parameter; any nonzero payload raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Identify
motor.identify();
```

Motion Control

Trapezoid move

Travel the given signed displacement over exactly the given duration, relative to the position at the end of previously queued motion (for an absolute target use 'Go to position'). The move is queued, not immediate: it is appended to the 32-item movement queue (normally exactly 3 slots: accelerate/coast/decelerate (a normal-length zero-displacement timed dwell still takes 3 because its accelerate and decelerate segments have nonzero durations even though their acceleration is zero), but the coast item is silently dropped whenever its computed duration works out to exactly zero -- which happens for any move whose duration is even and at most twice max velocity divided by max acceleration (a short even-duration triangular move, or an equally short dwell) -- leaving the move in only 2 slots) and begins only after all previously queued items finish. The success response confirms validation and queueing only, not motion. Speed follows from displacement/duration; the 'Set maximum velocity' and 'Set maximum acceleration' settings only size the acceleration ramp and act as hard limits, raising fatal error `ERROR_ACCEL_TOO_HIGH` or `ERROR_PREDICTED_VELOCITY_TOO_HIGH` when violated. Other fatal errors: `ERROR_QUEUE_IS_FULL`, `ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE`, `ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE`, `ERROR_MOTOR_BUSY` during calibration or homing, and `ERROR_COMMAND_SIZE_WRONG` for any payload other than 8 bytes. A duration of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (in firmware before 0.15.4.0 it was a silent no-op that still returned success). The profile assumes zero initial velocity: if the preceding queued motion ends at velocity v_0 , the actual displacement becomes the commanded value plus v_0 times the duration and the move ends at v_0 , not at rest. Accepted even with MOSFETs disabled, in which case the commanded position advances anyway and typically trips fatal error `ERROR_POSITION_DEVIATION_TOO_LARGE`.

Parameters:

- *displacement*: i32: The signed relative displacement to travel, measured from the position at the end of the previously queued motion (not an absolute position). Can be positive or negative. Internally in encoder counts; counts per rotation are motor-specific (see the unit conversion file), e.g. 3276800 counts per shaft rotation on M3/M17.
- *duration*: u32: The total time over which to perform the move, including the acceleration and deceleration ramps. Internally counted in 32 microsecond timesteps (31250 per second). A duration of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (silently ignored in firmware before 0.15.4.0).

Example:

```
// Trapezoid move
int32_t displacement = 1000;
uint32_t duration = 1000;

motor.trapezoidMove(displacement, duration);
```

Go to position

Queue a smooth absolute move. The firmware plans a trapezoid profile from the displacement and duration and adds three items (accelerate, coast, decelerate) to the 32-slot movement queue; the success response only acknowledges queueing and is sent before any motion occurs. Motion begins when earlier queue items finish. The target is planned against the predicted position at the end of everything already queued, not the current measured position, so successive calls chain correctly, but the reply never reflects actual arrival. Planning uses the maximum velocity and acceleration in effect at queue time, so send 'Set maximum velocity' and 'Set maximum acceleration' beforehand. The profile assumes the motor starts at rest: if the preceding queued motion ends at nonzero velocity (for example after 'Move with velocity') the accelerations superimpose and the motor does not stop at the requested position, so only issue this command when prior queued motion ends at rest. A duration of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (in firmware before 0.15.4.0 it was silently dropped with a success response). Fatal errors: 19 `ERROR_MOTOR_BUSY` if calibration or homing is running, 17 `ERROR_QUEUE_IS_FULL` if fewer than 3 slots are free, 15 `ERROR_ACCEL_TOO_HIGH` if the duration is too short for the distance, 28 `ERROR_PREDICTED_VELOCITY_TOO_HIGH`, 27 `ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE` or 26 `ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE` if the move or its overshoot point leaves the safety limits, and 46 `ERROR_MOVE_TOO_FAR` if the required displacement exceeds plus or minus 2^{31} counts. Broadcasting executes the move on all devices with no response, which is useful for synchronized multi-axis moves.

Parameters:

- *position*: i32: New absolute position value. The displacement is planned against the predicted position at the end of all items already in the movement queue, not the current measured position, so consecutive queued moves chain correctly.
- *duration*: u32: Time allowed for executing the move. A duration too short for the distance given the current maximum acceleration raises fatal error 15, `ERROR_ACCEL_TOO_HIGH`; a duration of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (silently dropped in firmware before 0.15.4.0).

Example:

```
// Go to position
int32_t position = 1000;
uint32_t duration = 1000;

motor.goToPosition(position, duration);
```


Homing

Home the motor by moving until it hits a hard stop. The command queues a trapezoid move of `maxDistance` over `maxDuration` and sets the homing and motor busy status flags; the success response is sent as soon as homing starts, not when it finishes. Because the move is paced across the FULL `maxDuration`, the approach speed is approximately `maxDistance` divided by `maxDuration` -- a generous `maxDuration` is not merely a timeout, it directly makes the approach gentler (bench-verified: 2 rotations with a 5 second budget crawled at ~0.4 rotations/second). A `maxDistance` of 0 is accepted and simply holds the homing/busy state for the full `maxDuration` without motion; a `maxDuration` of 0 is rejected with fatal error 34. Poll 'Get status' and wait for the homing flag (bit 4) and motor busy flag (bit 6) to clear. A collision is declared when the commanded and hall-sensor positions differ by more than 50,000 encoder counts (about 0.015 shaft rotations on M17/M23); the queue is then cleared and the commanded position snaps to approximately the stall position, leaving the motor holding there in closed loop with MOSFETs still enabled. If nothing is hit, the motor travels the full `maxDistance`; status flags cannot distinguish the two outcomes, so compare positions afterward.

Preconditions: closed-loop mode (else fatal error 13, `ERROR_NOT_IN_CLOSED_LOOP`), empty queue (else fatal error 8, `ERROR_QUEUE_NOT_EMPTY`), not busy (else fatal error 19, `ERROR_MOTOR_BUSY`). Normal enqueue-time planning checks apply, so a `maxDuration` too short for `maxDistance` or a move that would cross safety limits is fatal (errors 15, 28, 27, or 26). Homing does not zero the position and does not set safety limits; follow it with 'Zero position' and, if desired, 'Set safety limits'. If the max allowable position deviation has been set below 50,000 counts, a collision instead raises fatal error 45, `ERROR_POSITION_DEVIATION_TOO_LARGE`.

Parameters:

- *maxDistance*: i32: The maximum distance to move while searching for a hard stop (if no collision occurs, the motor travels this full distance). This can be positive or negative; the sign determines the direction of movement.
- *maxDuration*: u32: The time allotted for the homing move. Homing is queued as a trapezoid move covering `maxDistance` in this time, so a duration too short for the distance does not merely move the motor too fast: it raises fatal error 15, `ERROR_ACCEL_TOO_HIGH`, or 28, `ERROR_PREDICTED_VELOCITY_TOO_HIGH`, at enqueue time.

Example:

```
// Homing
int32_t maxDistance = 1000;
uint32_t maxDuration = 1000;

motor.homing(maxDistance, maxDuration);
```

Go to closed loop

Enter closed-loop position control mode. This command executes immediately (it is not queued) but has two preconditions: the movement queue must be empty, otherwise fatal error 8, `ERROR_QUEUE_NOT_EMPTY`, is raised, and the motor must not be busy, otherwise fatal error 19, `ERROR_MOTOR_BUSY`, is raised; on either fatal error the device replies with an error packet instead of a success response and stays in the fatal-error state until reset. The command automatically enables the MOSFETs if they are disabled (no separate 'Enable MOSFETs' is needed) and runs a current-sensor baseline check during that step, which can raise fatal error 22, `ERROR_CURRENT_SENSOR_FAILED`. It loads the commutation offset from saved settings without verifying that calibration was ever performed, so run 'Start calibration' at least once on a new unit. Completion semantics differ by product: on M2, M17, and M23 the transition is synchronous, so receiving the success response means the motor is already in closed loop (confirm via 'Get status' bit 2); on legacy M1 the success response only means an asynchronous procedure started, completion is indicated by status bit 5 clearing and bit 2 setting, and a failed attempt raises fatal error 39, `ERROR_GO_TO_CLOSED_LOOP_FAILED`. Calling it while already in closed loop is harmless on non-M1 products (given an empty queue and a non-busy motor). Any payload bytes raise fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Go to closed loop
motor.goToClosedLoop();
```

Move with acceleration

Queue a constant-acceleration segment: the acceleration is applied once per time step for the given number of time steps. This is a queued command; the success response only confirms the item entered the 32-item motion queue shared with 'Trapezoid move', 'Go to position', 'Move with velocity', and 'Multimove', and it executes after all previously queued items finish. WARNING: if the queue empties while velocity is non-zero, fatal error 18, `ERROR_RUN_OUT_OF_QUEUE_ITEMS`, is raised on the very next control tick, so every plan must end at zero net velocity (for example a mirror-image deceleration item) or be continuously fed with follow-up queue items. Enqueue-time validation can raise fatal errors 15 `ERROR_ACCEL_TOO_HIGH`, 28 `ERROR_PREDICTED_VELOCITY_TOO_HIGH`, 27 `ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE`, 26 `ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE`, 17 `ERROR_QUEUE_IS_FULL`, and 19 `ERROR_MOTOR_BUSY`; an error packet is sent instead of the success response on any fatal error and the device is left with MOSFETs disabled, answering only 'Get status' and 'System reset' until reset. This command does not enable the MOSFETs; with them disabled the profile still executes internally and the commanded position advances with no physical motion, so use 'Enable MOSFETs' or 'Go to closed loop' first. A `timeSteps` value of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (silently ignored in firmware before 0.15.4.0). Broadcast queues the move on every listening motor with no response, useful for synchronized multi-axis starts. The payload must be exactly 8 bytes or fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, is raised.

Parameters:

- *acceleration*: i32: The acceleration. The wire value is the acceleration in counts per time step per time step multiplied by 2^{24} (a fixed-point scale factor). It is applied once per time step; one time step is $1/\text{updateFrequency}$ seconds as reported by 'Get product specs'.
- *timeSteps*: u32: The number of time steps to apply this acceleration; query 'Get product specs' for the time step frequency. After this many time steps the acceleration ends, but the reached velocity is NOT held indefinitely: if the movement queue is empty when this item finishes at non-zero velocity, fatal error 18, `ERROR_RUN_OUT_OF_QUEUE_ITEMS`, is raised on the next control tick. End the plan at zero velocity or keep the queue fed. A value of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (silently ignored in firmware before 0.15.4.0).

Example:

```
// Move with acceleration
int32_t acceleration = 1000;
uint32_t timeSteps = 1000;

motor.moveWithAcceleration(acceleration, timeSteps);
```

Move with velocity

Rotate the motor at a constant velocity for a fixed duration. This is a queued command: it is appended to the 32-entry motion queue and executes only after all previously queued items finish; the success response acknowledges enqueueing, not motion, so the next command can be sent immediately. When the item executes, the velocity is applied as an instantaneous step (no ramp) and held constant for exactly the given duration. WARNING: if the queue is empty when the duration expires and the velocity is nonzero, the device raises fatal error 18, `ERROR_RUN_OUT_OF_QUEUE_ITEMS`, disabling the MOSFETs until 'System reset'; always queue a follow-up item that ends at zero velocity (for example 'Move with velocity' with velocity 0, or a decelerating 'Move with acceleration') before the duration expires. Enqueueing itself can instead raise fatal error 19, `ERROR_MOTOR_BUSY` (calibration, homing, or go-to-closed-loop in progress), 16, `ERROR_VEL_TOO_HIGH`, 27, `ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE`, or 17, `ERROR_QUEUE_IS_FULL`. A duration of zero is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (in firmware before 0.15.4.0 it was silently ignored with a success response). The queue is consumed even while the MOSFETs are disabled, so error 18 can fire with the motor unpowered. Commanding a velocity beyond what the motor can physically reach (about 8.6 rotations/second on an unloaded M17 -- measured identical at 20 V and 24 V across 40 units, so this ceiling is intrinsic to the drive, not the supply voltage) raises no error while the position deviation stays inside the allowed limit: the commanded position simply runs ahead, and in closed loop the motor keeps moving AFTER the queue empties until it catches up to the commanded end position -- an empty queue therefore does not mean motion has finished. One timestep is 32 microseconds (31250 timesteps per second). The velocity field is a SIGNED 32-bit wire value, which caps the largest commandable velocity at about 19.5 rotations/second in either direction -- values beyond that are rejected by client libraries before transmission.

Parameters:

- *velocity*: i32: The velocity. The raw wire value is the desired velocity in microsteps (counts) per timestep multiplied by 2^{20} , i.e. 12.20 fixed point; one timestep is 32 microseconds (31250 per second). A magnitude exceeding the configured maximum velocity raises fatal error 16, `ERROR_VEL_TOO_HIGH`.
- *duration*: u32: The time to maintain this velocity, in timesteps of 32 microseconds (31250 per second). A value of 0 is rejected with fatal error 34, `ERROR_PARAMETER_OUT_OF_RANGE` (silently ignored in firmware before 0.15.4.0).

Example:

```
// Move with velocity
int32_t velocity = 1000;
uint32_t duration = 1000;

motor.moveWithVelocity(velocity, duration);
```

Multimove

Enqueue up to 32 moves in one command; each move is an acceleration segment or an instant velocity change (selected per move by the moveTypes bitmask) executed for its given number of time steps. Two verified gotchas: an entry with 0 time steps is DROPPED SILENTLY and consumes no queue slot (unlike standalone move commands, which reject zero durations with fatal error 34 since firmware 0.15.4.0), and an acceleration-type entry with value 0 MAINTAINS the current velocity rather than stopping -- a trailing stop entry must be a velocity-type 0, or the plan ends at nonzero velocity and raises fatal error 18 when the queue empties. The success response only acknowledges that the moves were validated and enqueued, not that any motion completed. Moves share the single 32-entry motion queue with 'Trapezoid move', 'Go to position', 'Move with velocity', and 'Move with acceleration'; each consumes one queue spot except zero-time-step moves, which are silently discarded. Values use the same internal units as 'Move with acceleration' and 'Move with velocity'. Each move is validated at receive time and can raise fatal error 19 ERROR_MOTOR_BUSY, 15 ERROR_ACCEL_TOO_HIGH, 16 ERROR_VEL_TOO_HIGH, 28 ERROR_PREDICTED_VELOCITY_TOO_HIGH, 27 ERROR_PREDICTED_POSITION_OUT_OF_SAFETY_ZONE, 26 ERROR_TURN_POINT_OUT_OF_SAFETY_ZONE, or 17 ERROR_QUEUE_IS_FULL. If the queue empties while the commanded velocity is nonzero, the device raises fatal error 18, ERROR_RUN_OUT_OF_QUEUE_ITEMS; on emptying it also cross-checks predicted versus actual position and can raise fatal error 42, ERROR_POSITION_DISCREPANCY. A packet claiming more than 32 moves is rejected with fatal error 24, ERROR_MULTIMOVE_MORE_THAN_32_MOVES, before any move data is copied (firmware 0.15.4.0 and later; WARNING: firmware before 0.15.4.0 corrupted device memory before this error could fire).

Parameters:

- *moveCount*: u8: Specify how many moves are being communicated in this one shot. Must be 32 or fewer: a packet claiming more than 32 moves is rejected with fatal error 24, ERROR_MULTIMOVE_MORE_THAN_32_MOVES, before any move data is copied into the device buffer (the count has been validated ahead of the copy since firmware 0.10.0.0; only firmware older than that copied first and could corrupt device memory).
- *moveTypes*: u32: Bitmask selecting the type of each move, LSB first: bit *i* governs move *i*, where bit = 0 means a move with acceleration and bit = 1 means a move with velocity (same semantics as 'Move with acceleration' and 'Move with velocity').
- *moveList*: list_2d: A 2D list in Python format (list of lists). Each item is of type [i32, u32]: the first value is the acceleration to move at or the velocity to instantly change to (according to the moveTypes bits), the second is the number of time steps over which that move is executed. For example: '[[100, 30000], [-200, 60000]]'. Values use the same internal scaling as 'Move with acceleration' and 'Move with velocity'. There is a limit of 32 moves per command. Each move takes one spot in the shared 32-entry motion queue, except moves with 0 time steps, which are silently discarded and take no spot; make sure there is enough free queue space to store all of the moves.

Example:

```
// Multimove
uint8_t moveCount = 1;
uint32_t moveTypes = 1000;
uint32_t moveList = 1000;

motor.multimove(moveCount, moveTypes, moveList);
```

Other

Capture hall sensor data

Capture hall-sensor data and stream it back. This command is fully implemented: it starts a sampling engine in the motor-control interrupt and returns exactly one extended-size response packet whose payload bytes arrive incrementally over the whole capture duration ($nPointsToRead$ times $timeStepsPerSample$ times $nSamplesToSum$ time steps), with a single CRC32 at the end if CRC is enabled, so keep reading one packet until it completes. Warning: the device ignores all incoming packets until the capture finishes; there is no way to abort over the bus and only completion or a power cycle restores command processing, so queue any needed motion before sending this command. A fatal error during the capture (for example error 18 from a queued move that underruns mid-capture) kills the sampling engine and truncates the stream -- the host's read times out; send 'Get status' afterward to discover the latched code. Make sure all queued motion ends at rest before capturing. Unlike other commands it responds even when broadcast, so broadcasting with multiple devices on the bus causes collisions. All numeric parameters must be nonzero; invalid values raise fatal error 49, `ERROR_CAPTURE_BAD_PARAMETERS`, halting the device, which replies with an error packet (when addressed non-broadcast) and stays in the fatal-error state until 'System reset'. The total payload, $nPointsToRead$ times the number of enabled channels times 2 bytes, must not exceed 65526 bytes or fatal error 20, `ERROR_CAPTURE_PAYLOAD_TOO_BIG`, is raised. If sampling outruns the 230400-baud transmit drain the device halts with fatal error 21, `ERROR_CAPTURE_OVERFLOW`, so choose $timeStepsPerSample$ times $nSamplesToSum$ accordingly. For position capture (type 2) also mind ALIASING: each sample is a 16-bit value that wraps, so the position change between consecutive samples must stay below 32768 raw units or the unwrapped trace reads falsely slow -- at high speeds sample fast (e.g. $timeStepsPerSample$ 2, $nSamplesToSum$ 2 = 128 microseconds/sample tracks ~10 rotations/second safely). The raw position unit is not encoder counts; calibrate it against a known move. Sampling is skipped entirely while homing or calibration is active.

Parameters:

- *captureType*: u8: Type of data to capture: 1 = raw hall-sensor ADC readings (3 channels), 2 = fused hall position (a single value placed in channel slot 0 only, so use a channel bitmask of 1), 3 = midline-adjusted hall readings. Any other value, including 0, raises fatal error 49, `ERROR_CAPTURE_BAD_PARAMETERS`; there is no stop or abort value.
- *nPointsToRead*: u32: Number of points to read back from the device. Must be nonzero, and $nPointsToRead$ times the number of enabled channels times 2 bytes must not exceed 65526 (for example at most 10921 points with 3 channels or 32763 with 1).
- *channelsToCaptureBitmask*: u8: Channels to capture bitmask. Bits 0 to 2 enable hall sensors 1 to 3 respectively (bit value 1 enables the channel). The mask must be nonzero and bits 3 to 7 must be zero, otherwise fatal error 49, `ERROR_CAPTURE_BAD_PARAMETERS`. For captureType 2 only channel slot 0 receives data, so use a bitmask of 1; enabling higher bits just pads each point with zeros.
- *timeStepsPerSample*: u16: Acquire one sample every this many time steps. Time steps happen at the update frequency, which can be read with the 'Get product specs' command. Must be nonzero.
- *nSamplesToSum*: u16: Number of samples to sum together to make one point to transmit back. Must be nonzero. Together with *timeStepsPerSample* this sets the per-point period, which must be long enough for each point to transmit at 230400 baud before the next completes, or the device halts with fatal error 21, `ERROR_CAPTURE_OVERFLOW`.
- *divisionFactor*: u16: Division factor applied to each point's sample sum before transmission. Must be nonzero. The result is silently truncated to 16 bits, so choose it such that the maximum sample value times $nSamplesToSum$ divided by *divisionFactor* is at most 65535.

Example:

```
// Capture hall sensor data
uint8_t captureType = 1;
uint32_t nPointsToRead = 1000;
uint8_t channelsToCaptureBitmask = 1;
uint16_t timeStepsPerSample = 100;
uint16_t nSamplesToSum = 100;
uint16_t divisionFactor = 100;

captureHallSensorDataResponse response = motor.captureHallSensorData(captureType, nPointsToRead, channelsToCapt
```

Read multipurpose buffer

Read out and clear the multipurpose data buffer, which holds at most one dataset at a time. On current products (M3, M17, M23) it is filled only via 'Test mode': mode 3 stores a 20-byte PID debug snapshot (five packed i32: error, P term, I term, D term, output). Mode 3 is CONTINUOUS: it stores whenever the buffer is empty AND the PID is running (closed loop), so each read is immediately followed by a fresh snapshot -- back-to-back reads return new data, never the empty tag; in open loop the buffer stays empty. Mode 4 (GC6609 driver register dump) is DEFUNCT on current hardware: the GC6609 chip is used only on legacy M17 units (software compatibility code 1); on current units the code is not compiled in, so mode 4 stores nothing (the buffer stays empty and the test mode stays latched until reset). Also, switching to mode 4 while mode 3 is active can corrupt a pending dataset (tag-only response) -- use one test mode at a time and read the buffer before switching. Go-to-closed-loop and vibrate datasets are produced only on product M1. Contrary to older documentation, calibration never fills this buffer, and 'Capture hall sensor data' streams its data in its own response instead. The first byte of the returned data identifies the contents (2 = go-to-closed-loop, 3 = vibrate, 4 = PID debug, 5 = GC6609 registers), followed by the raw bytes. A successful read clears the buffer, so each dataset can be read exactly once. If the buffer is empty the device replies with a single data-type byte of 0 as of firmware 0.15.4.0; in older firmware it transmitted nothing at all, so the client timed out. A non-empty reply uses the extended-size packet format (first size byte 255, then a u16 total packet size), allowing up to 65535 bytes; the empty-buffer reply (the single 0 byte) arrives as a normal short-format packet. Broadcast produces no response and does not clear the buffer.

Example:

```
// Read multipurpose buffer
readMultipurposeBufferResponse response = motor.readMultipurposeBuffer();
```


Status & Monitoring

Get current time

Return the device's absolute time as an unsigned 64-bit count of true microseconds elapsed since power-up or since the last 'Reset time'; it is not wall-clock time. Executes immediately (not queued) and has no side effects. The wire value needs no scaling: it is a genuine 1 MHz microsecond count, effectively 48 bits wide, wrapping only after roughly 8.9 years. Silent on broadcast (no reply is sent). Any nonzero payload raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`. In rare cases of clock malfunction the read itself can raise fatal error 1, `ERROR_TIME_WENT_BACKWARDS`, which indicates a firmware or hardware fault rather than a usage error.

Example:

```
// Get current time
getCurrentTimeResponse response = motor.getCurrentTime();
```

Get n queued items

Return the number of items currently in the movement queue as a single byte between 0 and 32 (the queue holds at most 32 items). Executes immediately (not queued) and has no side effects; poll it to pace movement commands. This matters because queueing a movement command while the queue already holds 32 items is not a benign rejection: it raises fatal error 17, `ERROR_QUEUE_IS_FULL`, which disables the MOSFETs and halts the device until 'System reset'. Related pitfalls when reconciling this count with commands sent: queueing a move while the motor is busy (for example calibrating or homing) raises fatal error 19, `ERROR_MOTOR_BUSY`, and single-move commands with a duration of zero time steps are rejected with fatal error 34 as of firmware 0.15.4.0 (older firmware silently dropped them); zero-duration items inside a 'Multimove' are still silently dropped and never occupy a queue slot. Silent on broadcast (no reply is sent). Any nonzero payload raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Get n queued items
getNQueuedItemsResponse response = motor.getNQueuedItems();
```

Get hall sensor position

Get the position as measured by the hall sensors, which is the actual shaft position. It is reported in the same frame as the commanded position, so in normal operation it reads about the same as 'Get position'; if the two deviate by more than the max allowable position deviation (default 2 shaft rotations), the firmware raises fatal error 45, `ERROR_POSITION_DEVIATION_TOO_LARGE`. This is an immediate read-only query that returns exactly one response; the value is zeroed together with the commanded position by 'Zero position'. No response is sent when broadcast. Any payload bytes raise fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Get hall sensor position
getHallSensorPositionResponse response = motor.getHallSensorPosition();
```

Get status

Get the motor's status flags and fatal error code. This is an immediate query and one of only two commands still answered normally after a fatal error (the other is 'System reset'); every other command then receives an error packet until the device is reset. In normal operation the flags are recomputed at the moment of the request. In the fatal-error state the flags all read 0 as of firmware 0.15.4.0 (the MOSFETs are off and nothing is running); in firmware before 0.15.4.0 they were a frozen stale snapshot, so bit 1 could still read as MOSFETs-enabled and only the fatal error code was trustworthy. When broadcast, this command does nothing at all: no reply is sent and the stored snapshot is not even refreshed. No flag reflects ordinary queued motion (the busy flag covers only long-running tasks such as calibration and homing), so to detect motion completion poll 'Get n queued items' and confirm the position has settled rather than watching these flags. Sending any payload bytes raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Get status
getStatusResponse response = motor.getStatus();
```

Control hall sensor statistics

Turn hall sensor statistics gathering on or off. This command executes immediately (it is not queued). Sending 1 atomically resets the statistics (max = 0, min = 65535, sums = 0, count = 0) and turns gathering on; sending 0 turns gathering off, leaving the accumulated statistics frozen and readable via 'Get hall sensor statistics'. There is no way to reset without enabling, or to enable without resetting. Any control value other than 0 or 1 is silently ignored but still returns a success response. The payload must be exactly 1 byte; any other size raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`. Broadcast behavior is anomalous: when sent to the broadcast address 255 the command does nothing at all (most commands execute on broadcast and merely suppress the reply).

Parameters:

- *control*: u8: 0 = turn off statistics gathering (accumulated statistics remain frozen and readable), 1 = reset the statistics (max = 0, min = 65535, sums = 0, count = 0) and turn on gathering. Any other value is silently ignored but still returns a success response.

Example:

```
// Control hall sensor statistics
uint8_t control = 1;

motor.controlHallSensorStatistics(control);
```

Get hall sensor statistics

Read back the statistics gathered from the three analog hall sensors. This is useful for checking hall sensor health and the noise in the system. Executes immediately and returns an atomic snapshot (taken with the control-loop interrupt masked) of the per-sensor maximum, minimum, and sum, plus the total measurement count; no averages are returned, so compute $\text{average} = \text{sum} / \text{measurementCount}$ yourself. Reading does not stop or reset the statistics. Gathering is off by default after boot or reset and must be enabled with 'Control hall sensor statistics'; if this command is sent before gathering has ever been enabled since power-up, every field reads zero, including the minimums (0, not 65535), which can be misread as a dead sensor. Each recorded value is the sum of 4 oversampled 12-bit ADC readings, so the per-sample range is 0 to 16380, not 0 to 4095. Once measurementCount reaches 4294967295 the count and sums stop accumulating, while max and min continue to update. The payload must be empty; any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG. GOTCHA: after boot every field including the maxima reads 0 -- this means the statistics were never started (send 'Control hall sensor statistics' with 1 first), not that the sensors are dead. Healthy M17 baseline: across one full slow rotation each channel sweeps roughly 4000 to 12400 ADC units (peak-to-peak ~8200-8340); accumulation runs at ~31,400 samples/second.

Example:

```
// Get hall sensor statistics
getHallSensorStatisticsResponse response = motor.getHallSensorStatistics();
```

Get position

Get the current commanded (desired) position, i.e. the motion profile's internal target position that advances every control tick. This may differ slightly from the actual position measured by the hall sensors, which is available via 'Get hall sensor position'. Executes immediately (it is not queued), can be sent at any time including during motion, and takes an atomic snapshot with interrupts disabled. Commanded and measured position live in the same internal count space, so the same position unit conversion applies to both. The payload must be empty; any payload bytes raise fatal error 51, ERROR_COMMAND_SIZE_WRONG. Broadcast produces no response and has no side effects.

Example:

```
// Get position
getPositionResponse response = motor.getPosition();
```

Get comprehensive position

Get the commanded position, the hall sensor (measured) position, and the external encoder count in one shot. Executes immediately (not queued). The first two values are in encoder counts and convert normally with the motor's position units; the third is a raw signed count from an optional external quadrature encoder and must not be unit-converted (see its parameter note). The three values are read back-to-back rather than atomically as a set, so they can be skewed from one another by up to one control-loop tick. The external encoder count is never zeroed by any command, not even 'Zero position'; it accumulates from boot. Silent on broadcast: no response is sent. The payload must be empty or the device raises fatal error ERROR_COMMAND_SIZE_WRONG.

Example:

```
// Get comprehensive position
getComprehensivePositionResponse response = motor.getComprehensivePosition();
```

Get supply voltage

Get the measured voltage of the power supply. Executes immediately (not queued). The u16 reply is the supply voltage in decivolts (volts times 10); the firmware averages four ADC samples and applies a per-product calibration constant, so divide the value by 10 to get volts. Silent on broadcast: no response is sent. The payload must be empty or the device raises fatal error `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Get supply voltage
getSupplyVoltageResponse response = motor.getSupplyVoltage();
```

Get max PID error

Get the minimum and maximum position error observed by the closed-loop PID controller since the last read, then reset the accumulators. The error is commanded position minus hall sensor position in encoder counts, clamped to plus or minus 2^{29} before capture; a positive maximum means the measured position lagged behind the commanded position. Reading is destructive: it resets the accumulators to sentinel values (min = +2147483647, max = -2147483648), so two back-to-back reads return sentinels on the second read unless the PID loop ran in between. The error is only sampled while the PID controller actually runs (closed-loop position or velocity control); if the motor has not been in closed loop since the last read (or since boot), the reply is the sentinel pair itself. Always treat min greater than max as no data, not as a real error excursion (in practice this only appears if the device was never in closed loop during the window, because in closed loop the PID runs every 32-microsecond tick; an idle holding motor shows a dither of roughly plus/minus 500 counts). The wire values are encoder counts; client libraries that perform unit conversion, such as the Python library, return them converted into the currently selected position unit. On broadcast no reply is sent and the accumulators are not reset.

Example:

```
// Get max PID error
getMaxPidErrorResponse response = motor.getMaxPidError();
```

Get temperature

Get the temperature measured by a dedicated analog sensor on the motor driver PCB (not the microcontroller die and not the motor windings). The command executes immediately. Accuracy is about +/- 3 degrees Celsius, but only within the conversion table's range of roughly 33 to 307 degrees Celsius: any reading outside that range is reported as exactly 0, so a motor at room temperature reads 0 and negative temperatures are never reported despite the signed return type. Treat a value of 0 as an out-of-range sentinel, not a measurement. Independently of this command, the firmware raises fatal error 40, `ERROR_OVERHEAT`, at about 80 degrees Celsius, but only while the MOSFETs are enabled. The payload must be empty or fatal error 51, `ERROR_COMMAND_SIZE_WRONG`, is raised. When broadcast, no measurement is taken and no response is sent, so broadcasting this command is useless.

Example:

```
// Get temperature
getTemperatureResponse response = motor.getTemperature();
```

Get debug values

Get a snapshot of many internal diagnostic values. The snapshot contains: the maximum acceleration, maximum velocity, and current commanded velocity (all in raw internal units; no unit conversion is applied to any output of this command), the measured velocity, the number of time steps remaining in the currently executing queue item, four general-purpose debug values, execution-time profiler counters for the motor control loop, raw hall sensor voltages, the commutation position offset and phase-reversal flag, hall position delta statistics, and the motor PWM voltage. Executes immediately (not queued). Side effect: reading resets the hall position delta statistics, so `maxHallPositionDelta`, `minHallPositionDelta`, and `averageHallPositionDelta` cover only the interval since the previous read; if no samples accumulated in that interval, the max and min return the sentinel initialization values -2000000000 and +2000000000 and the average is meaningless. `debugValue1` to `debugValue4` are context-dependent diagnostic slots; when fatal error 45, `ERROR_POSITION_DEVIATION_TOO_LARGE`, trips they are overwritten with position-deviation forensics -- which are, however, `UNREACHABLE` in practice, because this command is not answered in the fatal-error state (it receives the error reply like every other command; verified on the bench). The profiler times are in microseconds; the motor-control calculations measure 17 us (open-loop idle) to 24-27 us (during a move) of the 32-microsecond control budget. Broadcasting this command does nothing at all: no reply is sent and the statistics are not reset. Any non-empty payload raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`.

Example:

```
// Get debug values
getDebugValuesResponse response = motor.getDebugValues();
```

Get communication statistics

Get six communication error counters and optionally reset them: CRC32 errors, packet decode errors (inconsistent packet size), first-bit errors (first byte of a packet lacked the required 1 in the least significant bit), and framing, overrun, and noise errors from the RS485 receiver. A nonzero reset flag clears all six counters, not just the CRC32 counter; the counters are snapshotted and cleared atomically in one call, so no events are lost between read and reset. Executes immediately (not queued). Counters are RAM-only and start at zero at every power-up or 'System reset'. Framing, overrun, and noise conditions are counted but are never fatal. This command is the only way to observe silently dropped packets: with CRC32 checking enabled (see 'CRC32 control'), a packet with a bad CRC32 is discarded with no error response and `crc32ErrorCount` increments. Broadcast gotcha: broadcasting this command does nothing at all, no reply and no reset, so counters cannot be bulk-reset over the bus. A payload that is not exactly 1 byte raises fatal error 51, `ERROR_COMMAND_SIZE_WRONG`. Verified counter-to-cause mapping: a first byte with its LSB clear increments `firstBitErrorCount`; a corrupted CRC32 increments `crc32ErrorCount` and is fully self-clearing (the very next command works, no resync wait needed); a TRUNCATED frame increments nothing immediately but jams the receiver until the 100 ms silence resync (measured threshold 95-105 ms), eating the next packet if sent too soon; an impossible declared size increments `packetDecodeErrorCount`.

Parameters:

- *resetCounter*: u8: Reset flag: 0 to just read; any nonzero value resets all six counters after they are read

Example:

```
// Get communication statistics
uint8_t resetCounter = 1;

getCommunicationStatisticsResponse response = motor.getCommunicationStatistics(resetCounter);
```